



Neural Network Assignments



Rakshya Lama Moktan • 1 Jul

Assignments • 100 points | Due 9 Jul, 20:45

You are a junior ML engineer at SmartForge Manufacturing. The production line images thousands of steel strips every day. Manual visual inspection is slow, inconsistent, and misses subtle defects that cause costly downstream failures. The engineering team needs two things: (1) a model that can identify which type of defect is present, and (2) evidence that the model is robust enough to handle real production variability (different lighting, orientations, and surface conditions). Your job is to build and harden that model using PyTorch.

Dataset

- NEU Surface Defect Database (Kaggle) — 1,800 grayscale images (200×200 px) of steel strip surfaces
- 6 defect classes: rolled-in scale (RS), patches (Pa), crazing (Cr), pitted surface (PS), inclusion (In), scratches (Sc)
- 300 images per class — balanced dataset
- Access:

kaggle.com/datasets/kaustubhdikshit/neu-surface-defect-database (load with `torchvision.datasets.ImageFolder`)

Part 0: NN Foundations

1.

Implement a simple 2-layer neural network from scratch using `nn.Module` — no `nn.Sequential` shortcuts. Define `__init__` and `forward` explicitly.

2.

Activation functions: swap ReLU for Sigmoid in your hidden layer and compare convergence over 20 epochs. What do you notice?



3.

Loss function choice: train with `nn.CrossEntropyLoss`.

Why is this preferred over `nn.MSELoss` for multi-class defect classification?

4.

Optimizer comparison: train the same network three times — `torch.optim.SGD`, SGD with momentum=0.9, and Adam. Plot all three loss curves on one graph. Which converges fastest? Estimate the GPU memory each optimizer requires for this model.

5.

Training stability: add `nn.BatchNorm1d` after your hidden layer OR `nn.Dropout(0.3)`. Re-train and report the effect on validation loss. Explain why the technique produces the effect you observe.

Part A: Defect Type Classifier

1.

Load the NEU dataset using `torchvision.datasets.ImageFolder` and `torch.utils.data.DataLoader`. How many images per class? What are the image dimensions?

2.

Normalise pixel values using `torchvision.transforms.Normalize`. Why is normalisation important before training a CNN?

3.

Build a CNN classifier using `nn.Module` with at least: `Conv2d` → `ReLU` → `MaxPool2d` (×2) → `Flatten` → Linear output for 6 classes.

4.

Write a full PyTorch training loop: `zero_grad()` → `forward()` → `CrossEntropyLoss` → `backward()` → `optimizer.step()`. Train for 15 epochs.

5.

Plot training vs. validation accuracy and loss. Is the model overfitting? At what epoch does it start?

6.

Per-class evaluation: which defect type has the lowest F1 score on the test set? Look at misclassified examples — why might crazing and patches be confused?



Part B: Model Hardening

The production line is messy.

Defects appear at any angle, under variable lighting, and at different positions in the frame. A model trained only on clean, centred images will fail in deployment.

7.

Data augmentation: add transforms for the training set only — `RandomHorizontalFlip`, `RandomRotation(15)`, `RandomCrop(180)`. Do NOT augment the validation/test set. Why?

8.

Batch normalisation: add `nn.BatchNorm2d` after each `Conv2d` layer. Retrain. How do the loss curves change compared to Part A? Explain the mechanism.

9.

Dropout: add `nn.Dropout(0.4)` before the final `Linear` layer. How does this affect overfitting? What is dropout doing during training vs. inference?

10. Plot

all three configurations (Part A baseline, +augmentation, +augmentation+BN+dropout) on the same validation accuracy graph. Which combination performs best?

11. Reflection:

if you could only apply one technique from this part to a new manufacturing dataset, which would you choose and why?



Part C: Hyperparameter Tuning

12. Grid

search: train your best model from Part B with all combinations of learning rate in {0.001, 0.01} and batch size in {16, 32}. Record validation accuracy for each of the 4 combinations.

13. Present

results as a markdown table. Which combination performed best? Which hyperparameter had more impact?

14. Learning

rate schedule: add `torch.optim.lr_scheduler.StepLR` (step every 5 epochs, $\gamma=0.5$) to your best run. Does it improve final test accuracy?

15. Extended

— Bayesian optimisation: use Optuna to search over lr ($1e-4$ to $1e-1$, log-uniform) and batch size (8 to 64). Does it find a better combination than your grid search?

NEU Surface Defect Databa...

<http://kaggle.com/datasets/kaustuk>



Your work

Assigned

[+ Add or create](#)

Mark as Done



Private comments

[Add comment to Rakshya Lam...](#)

Class comments

[Add comment](#)